

Know Unfold

A concise codebase and presentation study guide

Rajveer Singh Jolly

African Leadership University - Mobile Application Development

Unfold V1 Study Guide

The 20-second explanation

Unfold connects ALU students with student-led ventures. Students discover roles, turn a PDF CV into editable evidence, receive explainable skill matches, apply, and track progress. Founders publish opportunities and manage applicants. Flutter renders the app, Riverpod owns shared state, Firebase Auth identifies users, Firestore persists data, Firebase AI Logic analyzes the selected PDF, and security rules enforce ownership.

The architecture to memorize

view -> Riverpod controller -> repository interface -> Firebase

The view displays state and sends commands. The controller owns workflow state. The repository translates domain objects to backend operations. Firestore and Auth are replaceable implementations, which is why tests can use fakes.

Files you must know

File - What to say

`lib/main.dart` - Initializes Firebase, App Check, and Riverpod.

`lib/features/auth/session_controller.dart` - Observes auth, restores profiles, validates ALU domains, handles roles and sign-out.

`lib/features/opportunities/opportunity.dart` - Opportunity model and Firestore serialization, including compensation and deadline.

`lib/features/opportunities/opportunity_controller.dart` - Live feed, fallback data, bookmarks, applied IDs, submission, and refresh.

`lib/features/opportunities/opportunity_repository.dart` - Firestore opportunity create/read/update/delete.

`lib/features/opportunities/opportunity_match.dart` - Pure, deterministic skill-overlap calculation and related-role ranking.

`lib/features/applications/application.dart` - Application data and six status values.

`lib/features/applications/application_review_controller.dart` - Founder stream plus optimistic status update and rollback.

`lib/features/applications/application_repository.dart` - Firestore application queries and writes.

`lib/features/cv/cv_analysis_controller.dart` - PDF limit, Firebase AI Logic call, structured result, persistence, and editable skills.

`lib/features/home/views/discover_view.dart` - Search/filter UI plus cached search and matching.

`lib/core/widgets/match_ring.dart` - Static custom-painted score ring.

`lib/core/widgets/glass_surface.dart` - Reusable lightweight translucent surface.

`firestore.rules` - Backend authorization and deadline type validation.

One complete state flow

Application submission:

1. The student opens the form; no write occurs.
2. The form validates motivation and availability.
3. `OpportunityActivityController` creates the domain object.
4. The applied ID changes optimistically.
5. `FirestoreApplicationRepository` writes `applications/{studentId_opportunityId}`.
6. Rules verify the signed-in student, initial status, and correct opportunity owner.
7. The snapshot stream confirms the item.
8. On failure, the optimistic state rolls back and the UI displays an error.

Firestore schema

- `users/{uid}`: identity, profile fields, skills/interests, compressed photo, CV result.
- `opportunities/{id}`: owner, role, venture, skills, compensation, contact, optional timestamp deadline, status.
- `applications/{id}`: student/founder/opportunity IDs, answers, one of six statuses, timestamps.
- `bookmarks/{uid}/items/{opportunityId}`: private saved-role reference.

There is no active startup collection dependency in V1. Venture information is denormalized into an opportunity so the feed needs one read.

Deadline truth

The founder date picker supplies a `DateTime`. The repository converts it to Firestore data. When read, the model accepts a Firestore `Timestamp`. The UI derives words such as "Closes in 4 days" from that persisted value. Fallback roles have `deadline == null`, so they never show invented urgency.

CV intelligence and matching

The PDF remains in memory and is sent inline through Firebase AI Logic. It is limited to 8 MB. Gemini must return JSON containing skills, suggested roles, and a summary. Only those structured fields and the filename are stored in the user document; the raw PDF is not retained in Storage.

Matching is not another AI call:

```
score = matched required skills / total required skills
```

The user can remove extracted skills. That makes the evidence user-controlled and the score explainable. A score recommends discovery order; it never changes an application status.

Why scrolling is smoother in V1

The original match ring ran a 900 ms animation each time a list item rebuilt. Search normalization and skill overlap were also recalculated during parent rebuilds, and live backdrop blur was expensive on Android.

V1 fixes this by:

- painting a static match ring;
- caching search text and match objects until skills or opportunity data changes;
- isolating the rotating slogan timer in its own widget;
- replacing live navigation blur with static translucent layers.

Pull-to-refresh invalidates data providers. Scrolling does not fetch or recalculate percentages.

Security rules to memorize

- A profile is created only for the signed-in UID with an ALU-domain email.
- A student submits only as themselves and starts at submitted.
- The application's founder ID must match the live opportunity owner.
- Only the owner/admin changes application status; relationship IDs stay fixed.
- Opportunity create/update/delete requires authenticated ownership.
- A deadline is null/absent or a timestamp.
- A bookmark path is private to its UID.

Likely evaluator questions

Why Riverpod?

Shared auth, feed, CV, and application state outlive one widget. Riverpod makes dependencies explicit, supports streams and Notifiers, scopes rebuilds, and allows provider overrides in tests. Local visual input can still use `setState`.

Why Firebase?

Authentication, real-time Firestore listeners, security rules, and AI Logic cover the main backend needs without a custom server. Rules keep authorization beside the data.

Is hiding founder buttons secure?

No. The interface is convenience. Firestore rules check the authenticated owner even if a client calls the backend directly.

Why optimistic updates?

They make the interaction immediate. The repository remains the source of truth; a failed write restores the previous state.

Is AI deciding who gets hired?

No. AI extracts editable evidence. A pure function calculates overlap. Founders make every status decision.

Why no Cloud Storage?

The Spark-plan workflow did not require it. Profile photos are tightly compressed into the user record, and CV bytes are sent directly for analysis without raw-file retention.

What is original about Unfold?

It combines an ALU identity boundary, student/founder modes, compensation-aware publishing, editable CV intelligence, explainable matching, and an applicant pipeline in a mobile workflow shaped by physical-device feedback.

How would it scale?

Add pagination, indexed server-side filters, search infrastructure, notification functions, custom admin claims, App Check enforcement, analytics, and emulator-tested rules. The repository layer lets those changes occur without rewriting views.

Commands

```
flutter pub get
dart format --output=none --set-exit-if-changed lib test
```

```
flutter analyze  
flutter test  
flutter build apk --release --target-platform android-arm64
```

Presentation rule

Be exact. Show a working flow, name the controller and repository, show the matching Firebase document, explain the rule that protects it, and state one honest limitation. Understanding earns more confidence than memorized jargon.